# Blockchains and Distributed Ledgers : Smart Contract Programming Coursework Assignment Report

Royceton Yeoh Tze Jian (s2881515/B299013)

## 1 Detailed Contract Design Decisions

This Vickrey auction contract is designed to facilitate the sale of ERC721 NFTs in a sealed-bid, second-price auction format.

### 1.1 Bid Privacy During Bidding Phase

Bid confidentiality during the active bidding period is maintained using a cryptographic commit-reveal strategy.

1. **Commit Phase:** Bidders do not submit their actual bid value directly. Instead, they compute a hash (commitment) of their bid value combined with a secret nonce (a random string) using the keccak256 algorithm off-chain. This commitment hash is submitted to the contract along with a deposit (an amount of Ether greater than or equal to their intended bid). Because only the hash is stored on-chain during this phase, the actual bid values remain hidden from all participants, preventing information leakage that could influence other bids.

2. **Reveal Phase:** After the bidding deadline passes, a second phase begins where bidders reveal their original bid value and the secret nonce they used to create the commitment. The contract recalculates the hash on-chain using the revealed value and nonce. It then compares this recalculated hash with the commitment hash submitted earlier. A bid is only considered valid for winner determination if the revealed information correctly reproduces the original commitment hash. This two-phase process ensures bids are locked in during the first phase and cannot be altered based on subsequently revealed information.

### 1.2 Payment to Seller

To enhance security and robustness, the contract utilises the **Pull over Push** pattern for transferring the final payment to the seller.

1. Upon auction finalisation, the contract identifies the winning bidder and calculates the payment due to the seller. Following Vickrey rules, this payment is the greater of the second-highest valid bid revealed or the seller's predetermined reserve price (or just the reserve price if only one valid bid exists).

2. Rather than attempting an immediate, direct transfer (push) of Ether to the seller, which could fail or introduce security risks, the contract credits the calculated payment amount to an internal balance associated with the seller's address within the contract storage.

3. The seller must then actively initiate a separate transaction to withdraw these funds. This withdrawal function first checks the seller's available balance, resets this internal balance to zero (an essential step in the Checks-Effects-Interactions pattern), and proceeds to transfer the Ether to the seller's address.

This pull mechanism makes the payment process more reliable by separating the auction finalisation logic from the complexities and potential failures of external Ether transfers.

### 1.3 Prevention of Cheating (Seller/Bidder)

The contract incorporates several safeguards against malicious behaviour:

- **NFT Escrow:** The seller transfers the NFT into the contract's custody when starting the auction. The contract holds the NFT securely, preventing the seller from transferring it elsewhere during the auction. The NFT is only released to the winner upon successful finalisation or back to the seller if the auction concludes without a valid winning bid.
- **Seller Cannot Bid:** A specific check prevents the address that initiated the auction from participating as a bidder in their own auction. This acts as a direct countermeasure against shill bidding by the seller.
- **Bid Immutability (Commit-Reveal):** The commit-reveal process locks in a bidder's intention during the commit phase. They cannot change their bid amount after the bidding phase ends, nor can they selectively reveal a different bid than the one corresponding to their original commitment hash.
- **Single Bid Per Bidder:** The contract enforces that each bidder address can only submit one commitment per auction. Attempts to submit a second commitment from the same address will be rejected. This prevents strategies involving multiple bids from the same party.
- **Financial Commitment (Deposit Requirement):** Bidders must provide a deposit during the commit phase. During the reveal phase, the contract verifies that the revealed bid amount does not exceed the deposit provided. This ensures that any successfully revealed bid is backed by sufficient funds held in escrow by the contract.
- **Withdrawal Security (Re-entrancy Guard):** The functions handling fund withdrawals are structured to prevent re-entrancy attacks by adhering to the Checks-Effects-Interactions pattern, specifically updating internal balances before initiating external Ether transfers.

### 1.4 Tie Settlement

If multiple bidders reveal the identical highest bid amount, the contract resolves the tie based on the time of commitment. The bidder whose commitment transaction was recorded in an earlier block on the blockchain (indicated by a lower block number stored alongside their bid information) is declared the winner. This provides a deterministic and fair tie-breaking rule based on who committed their intention first.

### 1.5 Multiple Bids Handling

The contract design strictly prohibits a single bidder address from submitting more than one bid commitment within the same auction. This restriction is enforced during the commit phase by checking if the bidder already has a non-zero commitment hash stored for that specific auction. If a prior commitment exists, any subsequent commit attempts from that same address for that auction will fail, reverting the transaction.

### 1.6 Data Types and Structures

Appropriate data types and structures are used for clarity, efficiency, and safety:

- **Structs (‘Auction‘, ‘Bid‘):** Custom structures group related data logically. In this contract's case, all details pertaining to an auction and all details for a specific bid. This improves code readability and organisation.
- **Mappings (‘auctions‘, ‘bids‘, ‘pendingWithdrawals‘):** Mappings serve as key-value stores, providing efficient (constant-time complexity) lookups. `auctions` maps IDs to auction data, `bids` uses a nested structure (auction ID -> bidder address -> bid data) for direct access to any specific bid, and `pendingWithdrawals` tracks withdrawable balances per address.
- **Numerical Types (‘uint256‘):** The standard unsigned 256-bit integer is used for monetary values (Ether amounts are handled in Wei), token IDs, timestamps, durations, block numbers, and counters, aligning with EVM practices and providing a large range.
- **Addresses (‘address payable‘):** Addresses eligible to receive Ether (sellers and bidders) are marked as payable.
- **Hashes (‘bytes32‘):** The 32-byte type is used for storing commitment hashes, matching the output sise of the keccak256 algorithm.
- **Booleans (‘bool‘):** Used for simple status flags (e.g., whether an auction is active, finalised, or a bid revealed).
- **Interfaces (‘IERC721‘):** Using the standard interface type allows the auction contract to interact generically with any compliant ERC721 NFT contract for ownership checks and transfers.

These choices reflect standard Solidity practices, aiming for gas efficiency in common operations like data lookups while maintaining type safety and code clarity.

## 2 Gas Evaluation

### 2.1 Deployment and Interaction Costs

Transaction costs (in Gwei) based on observed Sepolia network interactions with a gas price of 1.500000011 Gwei

- Contract Deployment: 3739308.02492872 Gwei (Most expensive operation due to storing the contract bytecode on-chain).
- `startAuction`: 295918.501775511 Gwei (High cost due to multiple storage writes for new auction struct and the external NFT `safeTransferFrom` call).
- `commitBid`: 149859.00099906 Gwei (Primary cost from writing bidder's commitment and deposit to storage).
- `revealBid`: 168672.001236928 Gwei (Higher cost due to on-chain hash calculation, conditional winner logic, and potential multiple storage updates for auction winner/price fields).
- `finalisedAuction`: 191835.00166257 Gwei (High cost due to storage updates marking auction as finalised and the external NFT `safeTransferFrom` call to winner or seller).

- `claimLosingBid`: 70506.000517044 Gwei (Involves reading and updating storage for the specific bid and the bidder's withdrawable balance).
- `withdrawFunds`: 45487.500333575 Gwei (Involves storage update to zero balance and an external call to transfer Ether).

### 2.2 Fairness

Discuss gas fairness:

- **Auction Finalisation:** The `finalisedAuction` function can be called by anyone after the reveal period ends. The caller bears the gas cost for this potentially complex transaction (determining winner, calculating payment, transferring NFT). This could be seen as unfair if one party consistently has to trigger it. However, making it callable by anyone ensures the auction can be finalised even if the seller or winner is unresponsive.
- **Withdrawals:** The pull-pattern shifts the gas cost of receiving funds (`claimLosingBid` and `withdrawFunds`) entirely onto the beneficiaries (seller, refunded winner, losing bidders). While this adds steps and costs for users, it prevents burdening other functions with potentially unbounded loops or complex payout logic, which could lead to DoS. It can be argued this is fairer, as each user pays for their own withdrawal transaction.
- **Bidding Costs:** Gas costs for `commitBid` and `revealBid` should be relatively consistent for all bidders, promoting fairness during the core auction phases. Small variations might occur based on data sizes (e.g., zero vs non-zero storage writes), but significant systemic unfairness isn't apparent in these functions.

### 2.3 Cost Efficiency and Fairness Techniques

- **Pull-over-Push Pattern:** This enhances security and prevents unbounded gas costs in finalisation, shifting withdrawal costs to beneficiaries. This is a deliberate tradeoff favoring security and robustness over minimising transactions for users.
- **Data Structures:** Mappings provide efficient O(1) lookups, minimising gas for reading auction/bid data. Structs keep data organised without significant overhead.
- **Solidity Version:** Using `pragma solidity ^0.8.0;` leverages the built-in overflow/ underflow checks, which are generally more gas-efficient than importing and using external SafeMath libraries required in older versions.
- **Minimal Logic:** Functions generally perform only necessary actions. For instance, `finalisedAuction` doesn't attempt to refund all losers in a loop.

## 3 Potential Vulnerabilities, Hazards, and Mitigation

This section details potential security risks and the mechanisms implemented in the contract to mitigate them.

### 3.1 Identified Vulnerabilities/Hazards

Common smart contract vulnerabilities considered during design include:

- Re-entrancy attacks on withdrawal functions.
- Front-running during commit or reveal phases.

- Miner manipulation of timestamps (`block.timestamp`) affecting deadlines.
- Denial-of-Service (DoS) through gas limits or logic errors.
- Integer Overflow/Underflow bugs.
- Insufficient Access Control or Authorisation issues.
- Handling of incorrect or malicious inputs.

### 3.2 Security Mechanisms and Mitigation

The contract employs several strategies to address these potential issues:

- **Re-entrancy:** Mitigated primarily using the **Checks-Effects-Interactions pattern** in functions involving ETH transfer. Specifically, in `withdrawFunds`, the balance is zeroed (`pendingWithdrawals[msg.sender] = 0;`) before the external call (`msg.sender.call {value: amount}("")`). Similarly, in `claimLosingBid`, the deposit is zeroed (`bid.deposit = 0;`) before updating `pendingWithdrawals`. The overall **Pull-over-Push pattern** for all fund retrievals further minimises re-entrancy risks by isolating transfers into separate transactions initiated by the user.
- **Front-Running:** The **Commit-Reveal Scheme** protects bid values, as only hashes are visible during bidding. The commit transaction itself remains vulnerable to front-running for griefing (forcing a user to recommit) or attempts to claim the bid slot, although the bid value remains secret. Front-running the reveal does not alter the outcome based on bid value, as the bidding phase is closed.
- **Timestamp Dependence:** Deadlines (`biddingEndTime`, `revealEndTime`) rely on `block.timestamp`. While miners have minor control over timestamps, significant manipulation is difficult and costly. Using timestamps for deadlines is standard practice. The durations are set relative to the start time, providing a predictable window. The risk is considered low for typical auction durations.
- **Gas Limit Issues (DoS):** The pull pattern for withdrawals (`claimLosingBid` and `withdrawFunds`) avoids loops over arrays of bidders, which could otherwise lead to unbounded gas costs exceeding block limits as the number of bidders grows. This prevents DoS attacks aimed at making payouts impossible. Functions such as `revealBid` have bounded complexity.
- **Integer Overflow/Underflow:** The contract uses `pragma solidity ^0.8.0;`. Versions 0.8.0 and above include built-in checks for arithmetic overflow and underflow, automatically reverting the transaction if such conditions occur. This eliminates the need for external SafeMath libraries for standard arithmetic operations.
- **Access Control:** Multiple `require` statements enforce rules: only the NFT owner can start the auction (`nftContract.ownerOf(_tokenId) == msg.sender`), sellers cannot bid (`msg.sender != auction.seller`), bids/reveals must happen within the correct phases, reveals must match commitments (`bid.commitment == commitment`), winners cannot use `claimLosingBid` (`msg.sender != auction.highestBidder`), etc.
- **Incorrect Inputs:** `require` statements validate critical inputs like positive auction durations, non-zero deposits (`msg.value > 0`), and non-zero commitments (`_commitment != bytes32(0)`).

- **Single Commit Enforcement:** The contract prevents multiple commits per bidder per auction using `require(bids[_auctionId][msg.sender].commitment == bytes32(0), ...)`.
- **Deposit Requirement:** The check `require(bid.deposit >= _bidAmount)` ensures revealed bids are financially backed, preventing bidders from revealing unaffordable amounts.

# 4 Tradeoffs and Choices

Several design decisions involved balancing competing factors like security, user experience, gas efficiency, and fairness.

- **Security vs. User Experience (Pull vs. Push):** The most significant tradeoff is the use of the pull-over-push pattern for all ETH withdrawals (`withdrawFunds`, `claimLosingBid`). This pattern significantly enhances security against re-entrancy attacks and potential DoS issues related to failed transfers or gas limits if the contract were to push funds automatically. However, it degrades user experience as it requires beneficiaries (seller, refunded winner, losing bidders) to send 1-2 additional transactions (`claimLosingBid` if applicable, then `withdrawFunds`) to retrieve their ETH, incurring extra gas costs and steps. A push pattern would be more convenient but riskier. Security was prioritised here.
- **Fairness vs. Efficiency (Gas Costs):** The pull pattern shifts the gas cost of withdrawals to the users receiving funds. This can be seen as fairer than having the `finalisedAuction` caller or the contract potentially pay for multiple transfers, but it does impose costs on users. The fact that anyone can call `finalisedAuction` is efficient in ensuring finalisation but means the caller bears that gas cost, which might not always be the seller or winner. The tie-breaking mechanism using `commitBlock` is computationally simple and efficient, avoiding potentially more complex or gas-intensive methods, while providing a deterministic outcome.
- **Simplicity vs. Feature Richness:** The contract implements the core Vickrey auction logic with commit-reveal but omits features like auction cancellation, automatic extensions if bids occur near the end, proxy bidding, or more complex dispute resolution. This focus on simplicity reduces the potential attack surface and contract complexity, potentially enhancing security and auditability, at the cost of fewer features.
- **On-chain vs. Off-chain Logic:** All essential logic determining the auction state, validity of bids, winner, and payment amounts is performed on-chain. This ensures transparency and trustlessness, as all participants can verify the auction's execution based on blockchain data. The only off-chain components are the user-side calculation of the commitment hash and the monitoring of auction phases. Relying more heavily on off-chain components could potentially reduce gas costs but would likely compromise transparency or require additional trust assumptions.

# 5 Analysis of Fellow Student's Contract

## 5.1 Contract Address

Fellow Student's Sepolia Contract Address: [0xC8269b326055480513cB655a03d22963e4D342f4]

### 5.2 Security, Efficiency, Fairness Analysis

**Security:**

- **Re-entrancy Protection:** Uses OpenZeppelin's `ReentrancyGuard` (`nonReentrant` modifier) on key state-changing functions (`startAuction`, `endAuction`, `withdraw`). The `withdraw` function also follows Checks-Effects-Interactions by deleting the bid record (`delete committedBids[_auctionId][msg.sender];`) before the external ETH transfer (`msg.sender.safeTransferETH(_collateral);`). The `endAuction` function performs state updates (`active = false;`, `auctionId++;`, `committedBids[finishedId][highestBidder].collateral = difference;`) before external calls.

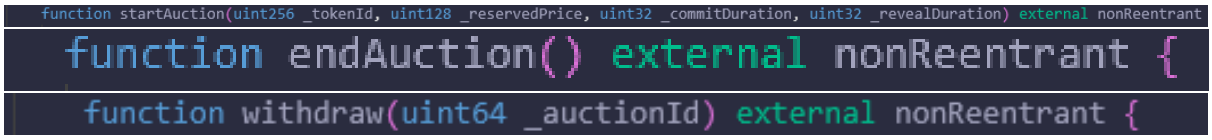


Figure 1: Code Snippet of nonReentrant modifier

- **Overflow/Underflow:** Use of `pragma solidity ^0.8.19;` provides automatic protection. Using `uint128` for amounts/collateral provides an additional layer against excessively large values.
- **Input Validation:** Strong validation exists in `startAuction` (ownership, approval, durations, reserve price). `commitBid` checks auction phase, non-zero hash, and commit limit. `revealBid` checks phase, committed bid existence, reveal status, hash matching, and deposit coverage (`if(bid.collateral >= amount){...}`).

```
require(!active, "VK_AU: Auction is already active, cannot start new one!");
require(_isOwnerOf(_tokenId, msg.sender), "VK_AU: Seller does not own target NFT!");
require(_isApproved(_tokenId), "VK_AU: Seller has not approved contract for transfer!");
require(_reservedPrice > 0, "VK_AU: Reserved Price cannot be zero!");
require(_commitDuration >= 1 minutes, "VK_AU: Bid Commit Period must be at least 1 minutes long!");
require(_revealDuration >= 1 minutes, "VK_AU: Bid Reveal Period must be at least 1 minutes long!");
require(_commitDuration <= 60 days, "VK_AU: Bid Commit Period cannot be longer than 60 days!");
require(_revealDuration <= 60 days, "VK_AU: Bid Reveal Period cannot be longer than 60 days!");
```

Figure 2: Code Snippet of input validation in startAuction function

- **Commit-Reveal:** Correctly implemented using `EfficientHashLib.hash` including bidder address, amount, nonce, token ID, and auction ID, preventing hash collisions across different contexts. Reveal logic correctly compares recomputed hash with stored `bidHash`.

```
bytes32 _bidHash = bid.bidHash;
require(_bidHash != bytes32(0), "VK_AU: No bid commitment found!");

bytes32 bidHash = EfficientHashLib.hash(uint160(msg.sender), amount,nonce, auction.tokenId, auctionId);
require(_bidHash == bidHash, "VK_AU: Invalid Bid Hash!");
```

Figure 3: Code Snippet of Hash check

- **Access Control:** NFT ownership/approval checked in `startAuction`. `commitBid` prevents the seller bidding (`require(msg.sender != auction.seller, ...)`). Phase timing is enforced.

- **Safe ETH Transfer:** Uses Solady's `SafeTransferLib` (`safeTransferETH`) for transfers in `endAuction` and `withdraw`, reducing risks associated with raw calls or `.transfer()`.

**Efficiency:**

- **Struct Packing:** Good use of smaller types (`uint128`, `uint32`, `uint8`, `bool`) aims to reduce storage costs. `CommittedBid` fits in 2 slots. `Auction` fits in 5 slots.

```
struct Auction {
    // 1 SLOT
    address seller;
    uint32 startOfAuction;
    uint32 endOfBidCommitTime;
    uint32 endOfBidRevealTime;
    // 1 SLOT
    address highestBidder;
    // 1 SLOT
    uint256 tokenId;
    // 1 SLOT
    uint128 highestBid;
    uint128 secondHighestBid;
    // 1 SLOT
    uint128 reservedPrice;
    bool finalized;
}
```

```
struct CommittedBid {
    // 1 SLOT
    bytes32 bidHash;
    // 1 SLOT
    uint128 collateral;
    uint8   bidNumber;
    bool    revealed;
}
```

Figure 4: Code Snippet of struct CommitedBid and Auction

- **Hashing Library:** `EfficientHashLib` potentially saves gas over standard hashing.
- **Auctions Array:** Using `Auction[] public auctions;` and incrementing `auctionId` only when ending an auction means only the latest auction is directly accessible via `auctions[auctionId] during its active phase`. Accessing past auctions requires iterating or knowing the specific index. `auctions.push()` gas cost increases over time. Managing only one active auction simplifies state but differs from typical multi-auction platforms. Use of `delete committedBids[_auctionId][msg.sender];` reclaims some gas upon withdrawal.

```
Auction[] public auctions;
```

Figure 5: Code Snippet of Auction[] public auctions Array

**Fairness:**

- **Gas Costs in `endAuction` (Push Pattern):** The caller of `endAuction` bears gas costs for ETH transfers to the seller and potentially the winner (refund), plus the NFT transfer. This contrasts with a pull pattern and can disincentivise calling the function, potentially delaying settlement.


Figure 6: Code Snippet of safeTransferETH in end Auction

- **Multiple Commits:** Allows up to 3 commits (`bid.bidNumber < 3`). Collateral is cumulative (`bid.collateral += uint128(msg.value);`), but only the last hash (`bid.bidHash = bidHash;`) matters for reveal. This might allow tactical deposit increases but primarily seems to add complexity and potential gas usage without clear benefit over a single commit model. It doesn't seem inherently unfair.

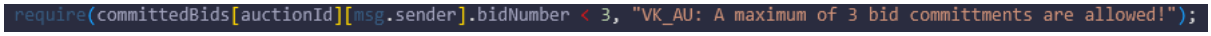


Figure 7: Code Snippet of maximum of 3 commits logic

- **Tie-breaking (Reveal Time):** Logic `if(amount > auction.highestBid)` implies the first reveal wins ties for the highest bid, as subsequent equal bids won't overwrite `auction.highestBidder`. This differs from using commit time but is still deterministic.

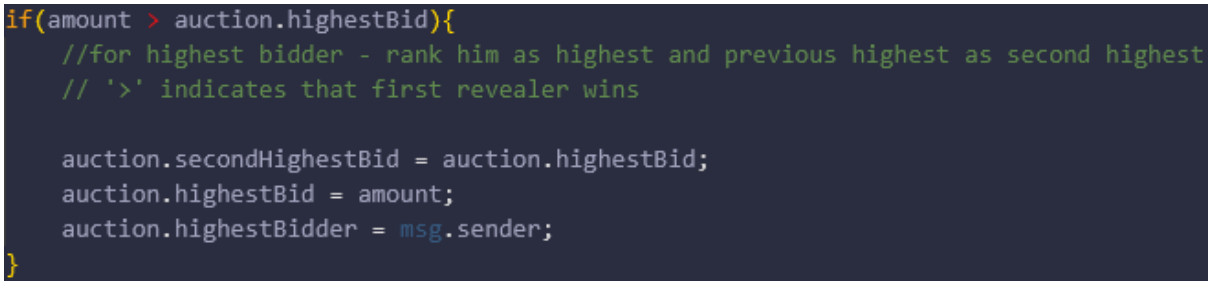


Figure 8: Code Snippet of reveal time tie breaking

- **Withdrawal Gas:** Losers and the winner (for refund difference handled in `endAuction`) call `withdraw` paying their own gas, which is fair.

### 5.3 Vulnerabilities Discovered

No critical security flaws were found. Areas for consideration:

- **Gas Burden / Potential Delay for `endAuction`:** The push payments within `endAuction` create a variable and potentially high gas cost for the caller, which could lead to delays if no party is incentivised to trigger it promptly. This is more a design choice trade-off than a vulnerability allowing theft.
- **Complexity of Multiple Commits:** Allowing multiple commits increases the contract's state size (`bidNumber`) and logic (`require(..., bidNumber < 3)`), adding slight gas overhead and complexity without a clear strategic advantage for bidders (since only the last hash counts). It could potentially be used for minor griefing.
- **Auction Array Growth:** The unbounded `auctions.push()` could eventually lead to very high gas costs for starting new auctions, though this requires a huge number of auctions to become problematic.

# 6 Contract Addresses and Transaction Hashes

## 6.1 Contract Address

My Sepolia Contract Address: [0xbCabCa54D6d26c19AFe4078C6e20eaFfd8B1716b]

## 6.2 Interaction Transaction Hashes

Interaction for Address(My contract):
[0xbCabCa54D6d26c19AFe4078C6e20eaFfd8B1716b]

| Actor | Action | Transaction Hash |
|---|---|---|
| Me | `startAuction` | 0x660d17ae5157ad213c9b38a2b485a4e8e6a594b2b6c2f6987c30a7aea34dcc0a |
| Me (Failed) | `commitBid` | 0xadd6a9d9f9ca2afcf2de0c8bfb6acde52b8ef0557e1c18cd6225391c167601f1 |
| Fellow Student | `commitBid` | 0xbd8d51ef0d60c66a2e4d9b44d4b5050a1a56c7e0d126bc95e167d0f0b941ccfe |
| Fellow Student | `revealBid` | 0x6a757b155b1cb4a5127c17659496b71c544cca1e4e248b860a9d4d902e64d958 |
| Fellow Student | `finalisedAuction` | 0x44bdfd51a198a071442366cfff4dfed5bf884a417793f47e0ab96e27c217aeed |
| Fellow Student | `withdrawFunds` | 0x4139420cc6402a37095417f8f58c103d21a7a5086e6b39e9c585bb54a9831f5d |

Interaction for Address(Fellow Student's contract):
[0xC8269b326055480513cB655a03d22963e4D342f4]

| Actor | Action | Transaction Hash |
|---|---|---|
| Fellow Student | `startAuction` | 0x18bc588de6caa8751f03206e75c6005f04358f532f6e7c28f90f4c6371e155d1 |
| Me | `commitBid` | 0xa6e7336d7a01e54ac826367186b0673c0a7056aea5e432a4eebd615e0d12eeb4 |
| Fellow Student | `commitBid` | 0xf721195c9f18d243c2bd05e29a70fac35af9c93dc0ab19bfec5db20c5ad216d4 |
| Me | `revealBid` | 0x0ec506c83bcf8338de3df838fad7996eec4b130a4003cc7d588f48cc6e825ef0 |
| Fellow Student | `revealBid` | 0x04e4f7030cc54bcbee8135728a7f3686edc4763f35781f42f770ff4eea245dc2 |
| Me | `endAuction` | 0x13359bf9634f50c2890786d5566f0e5dd0fb9021f20fde0e9837b3d2c7dcb792 |
| Me | `withdraw` | 0xf51694fea8b36e17d8d35725b3dd8c36fc192c5d0cd6eecc2daa7478d5f05276 |
| Fellow Student | `withdraw` | 0xf2b1d005abed83cb4bfc1746cd664965f9187251e5af94a433743fe6fb1a56c7 |

# 7 Contract Code

```
/// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// Imports the standard interface for an ERC721 NFT.
import "@openzeppelin/contracts/token/ERC721/IERC721.sol";

// Imports the interface required for the contract to safely receive an
    NFT.
import "@openzeppelin/contracts/token/ERC721/IERC721Receiver.sol";

// This contract implements the receiver interface, allowing it to hold
    an NFT during the auction.
contract VickreyAuction is IERC721Receiver {

    // State Variables
    // Stores the address of the ClassNFT contract. It's set once and
       unchangeable.
    IERC721 public immutable nftContract;

    // A custom data structure to hold all information about a single
       auction.
    struct Auction{
        address payable seller;         // The person selling the NFT.
        uint256 nftTokenId;             // The unique ID of the NFT
           being sold.
        uint256 reservePrice;           // The minimum price the seller
            will accept.
        uint256 biddingEndTime;         // The timestamp when the
           commit/bidding phase ends.
        uint256 revealEndTime;          // The timestamp when the
           reveal phase ends.
        bool isActive;                    // The flag to check activity
            of the auction.
        bool isFinalised;                 // The flag to check whether
           auction has been finalised.
        address payable highestBidder;  // The current winning bidder.
        uint256 highestBid;             // The highest bid amount
           revealed so far.
        uint256 secondHighestBid;       // The second-highest bid
           amount revealed.
        uint256 highestBidCommitBlock;  // The block number of the
           highest bid, used for tie-breaking.
    }

    // A custom data structure to hold a bidder's information for an
       auction.
    struct Bid{
        bytes32 commitment;            // The user's hashed bid (
           keccak256(amount, nonce)).
        uint256 deposit;               // The total Wei the bidder
           locked up with their bid.
        uint256 commitBlock;           // The block number when the bid
            was committed.
        bool isRevealed;                 // The flag to check whether
           bid was revealed.
    }
```

```
// A counter to give each new auction a unique ID. Not using
   counters library since version ^0.8 prevents underflow/overflow
uint256 public auctionCounter;

// Mappings
// A mapping from an auction's ID to its Auction data.
mapping(uint256 => Auction) public auctions;

// A nested mapping from an auction ID to bidder's address to
   bidder's Bid data.
mapping(uint256 => mapping(address => Bid)) public bids;

// A mapping to track how much Wei each address is owed from
   auctions.
// This is for the "pull" withdrawal pattern, which is more secure.
mapping(address => uint256) public pendingWithdrawals;

// Events
// Logs the creation and key details of a new auction.
event AuctionStarted(
    uint256 indexed auctionId,
    address indexed seller,
    uint256 indexed nftTokenId,
    uint256 reservePrice,
    uint256 biddingEndTime,
    uint256 revealEndTime
);

// Logs that a bidder has submitted their hidden (committed) bid.
event BidCommitted(
    uint256 indexed auctionId,
    address indexed bidder,
    bytes32 commitment
);

// Logs that a bidder has successfully revealed their bid amount.
event BidRevealed(
    uint256 indexed auctionId,
    address indexed bidder,
    uint256 amount
);

// Logs the final outcome of an auction, including the winner and
   final price.
event AuctionFinalised(
    uint256 indexed auctionId,
    address winner,
    uint256 paymentAmount
);

// Logs a successful withdrawal of funds by a specific user.
event FundsWithdrawn(
    address indexed user,
    uint256 amount
);

// Functions
```

```
// The constructor runs once when the contract is deployed.
// It permanently sets the address of the ClassNFT contract.
constructor(address _nftContract) {
    // Pass the address of ClassNFT.sol (0
       x1546Bd67237122754D3F0CB761c139f81388b210)
    require(_nftContract != address(0), "Invalid NFT contract
       address.");
    nftContract = IERC721(_nftContract);
}

// This function is required by IERC721Receiver. It allows the
   contract
// to accept an NFT from the safeTransferFrom function.
function onERC721Received(
    address operator,
    address from,
    uint256 tokenId,
    bytes calldata data
) external override returns (bytes4) {
    return this.onERC721Received.selector;
}

// Creates a new auction.
function startAuction(
    uint256 _tokenId,          // The ID of the NFT to be auctioned
        off.
    uint256 _reservePrice,     // The minimum price.
    uint256 _biddingDuration,  // How long the bidding phase lasts
       (seconds).
    uint256 _revealDuration    // How long the reveal phase lasts (
       seconds).
) external {
    // Checks that the person starting the auction is the real
       owner of the NFT.
    require(nftContract.ownerOf(_tokenId) == msg.sender, "Not the
       NFT's owner.");
    require(_biddingDuration > 0, "Bidding duration must be more
       than 0.");
    require(_revealDuration > 0, "Reveal duration must be more than
        0.");

    // Checks that the contract is approved to take the NFT.
    require( nftContract.getApproved(_tokenId) == address(this)
        || nftContract.isApprovedForAll(msg.sender, address(this)),
        "Contract not approved for this NFT."
    );

    // Transfers the NFT from the seller into this contract, which
       acts as a secure escrow.
    nftContract.safeTransferFrom(msg.sender, address(this),
       _tokenId);

    // Sets up the new auction's data in storage.
    uint256 auctionId = auctionCounter;
    Auction storage newAuction = auctions[auctionId];

    newAuction.seller = payable(msg.sender);
    newAuction.nftTokenId = _tokenId;
```

```
        newAuction.reservePrice = _reservePrice;
        newAuction.biddingEndTime = block.timestamp + _biddingDuration;
        newAuction.revealEndTime = newAuction.biddingEndTime +
            _revealDuration;
        newAuction.isActive = true;
        newAuction.isFinalised = false;

        // Increments the counter so the next auction gets a new ID.
        auctionCounter++;

        emit AuctionStarted(
            auctionId,
            newAuction.seller,
            newAuction.nftTokenId,
            newAuction.reservePrice,
            newAuction.biddingEndTime,
            newAuction.revealEndTime
        );
    }

    // Allows a bidder to commit their hashed bid and deposit.
    function commitBid(
        uint256 _auctionId,  // The ID of the auction to bid on.
        bytes32 _commitment  // The hashed bid (keccak256(amount, nonce
            )).
    ) external payable {
        Auction storage auction = auctions[_auctionId];

        // Checks to ensure the auction is in a valid state for bidding
            .
        require(msg.sender != auction.seller, "Seller cannot bid.");
        require(auction.isActive, "Auction not active.");
        require(block.timestamp < auction.biddingEndTime, "Bidding
            phase over.");

        // Check to ensure a bidder can only commit once.
        require(bids[_auctionId][msg.sender].commitment == bytes32(0),
            "Bid has already been committed.");

        require(msg.value > 0, "Deposit must be more than 0");
        require(_commitment != bytes32(0), "Commitment cannot be zero")
            ;

        // Stores the bidder's information.
        Bid storage newBid = bids[_auctionId][msg.sender];
        newBid.commitment = _commitment;
        newBid.deposit = msg.value;         // msg.value is the amount
            of Wei sent.
        newBid.commitBlock = block.number;  // Stores the block number
            for tie-breaking.
        newBid.isRevealed = false;

        emit BidCommitted(_auctionId, msg.sender, newBid.commitment);
    }

    // Allows a bidder to reveal their actual bid amount and nonce.
    function revealBid(
        uint256 _auctionId,  // The auction ID.
```

```
    uint256 _bidAmount,  // The real bid amount.
    bytes32 _nonce       // The secret random string.
) external {
    Auction storage auction = auctions[_auctionId];
    Bid storage bid = bids[_auctionId][msg.sender];

    // Checks to ensure the auction is in the correct reveal phase.
    require(auction.isActive, "Auction not active.");
    require(block.timestamp > auction.biddingEndTime, "Bidding
       phase not over.");
    require(block.timestamp < auction.revealEndTime, "Reveal phase
       over.");

    // Checks that the bidder has a committed bid to reveal.
    require(bid.commitment != 0, "No bid committed");
    require(!bid.isRevealed, "Bid already revealed");

    // Recalculate and check the hash to see if it matches the one
       committed earlier.
    bytes32 commitment = keccak256(abi.encodePacked(_bidAmount,
       _nonce));
    require(bid.commitment == commitment, "Invalid reveal");

    // Check if the revealed amount is covered by their deposit.
    require(bid.deposit >= _bidAmount, "Deposit less than required
       amount");

    // Marks the bid as revealed to prevent revealing again.
    bid.isRevealed = true;

    // Check if bid meet reserve price threshold.
    if(_bidAmount < auction.reservePrice){
        emit BidRevealed(_auctionId, msg.sender, _bidAmount);
        return; //Exit the function early
    }

    // Winner Determination Logic
    // If current bid is the new highest
    if(_bidAmount > auction.highestBid) {
        // The old highest bid becomes the new second-highest.
        auction.secondHighestBid = auction.highestBid;
        // This bid becomes the new highest.
        auction.highestBid = _bidAmount;
        auction.highestBidder = payable(msg.sender);
        auction.highestBidCommitBlock = bid.commitBlock;

    // If this bid is tied with the current highest
    } else if (_bidAmount == auction.highestBid) {

        // TIE-BREAKING RULE: Check if this bid was committed in an
            earlier block.
        if (bid.commitBlock < auction.highestBidCommitBlock) {
            // Current bid wins the tie breaking
            auction.secondHighestBid = auction.highestBid; // The
               tied amount is now second-highest.
            auction.highestBidder = payable(msg.sender);
            auction.highestBidCommitBlock = bid.commitBlock;
        } else {
```

```
            //Old bid comes first, the current bid is now second
               highest
            auction.secondHighestBid = _bidAmount;
        }
    // If current bid is not the highest, but is higher than the
       current second-highest
    } else if (_bidAmount > auction.secondHighestBid) {
        // Current bid becomes the new second-highest bid.
        auction.secondHighestBid = _bidAmount;
    }

    emit BidRevealed(_auctionId, msg.sender, _bidAmount);
}

// Finalises the auction, calculates the winner and price, and
   handles refunds.
function finalisedAuction(
    uint256 _auctionId  // The auction ID to end.
) external {
    Auction storage auction = auctions[_auctionId];

    // Checks to ensure the auction can be finalised.
    require(auction.isActive, "Auction not active.");
    require(block.timestamp > auction.revealEndTime, "Reveal phase
       not over.");
    require(!auction.isFinalised, "Auction already finalised");

    // Marks the auction as finished so this can't be run again.
    auction.isFinalised = true;
    auction.isActive = false;

    address payable winner = auction.highestBidder;
    uint256 paymentAmount = 0;

    // Check if there was a winner, at least one valid bid above
       reserve.
    if (winner != address(0)){

        // Second-Price Logic
        if (auction.secondHighestBid == 0) {
            // If only one bid, the winner pays the reserve price.
            paymentAmount = auction.reservePrice;
        } else {
             // If multiple bids, winner pays  the max between
                second-highest and reserve price.
            paymentAmount = (auction.secondHighestBid > auction.
               reservePrice) ? auction.secondHighestBid : auction.
               reservePrice;
        }

        // Credit the seller's internal account. They can withdraw
           this later.
        pendingWithdrawals[auction.seller] += paymentAmount;

        // Calculate the winner's refund (Deposit - Price Paid).
        uint256 winnerDeposit = bids[_auctionId][winner].deposit;
        uint256 winnerRefund = winnerDeposit - paymentAmount;
```

```
        // If the winner is owed a refund, credit their internal
           account.
        if (winnerRefund > 0){
            pendingWithdrawals[winner] += winnerRefund;
        }
    }

    emit AuctionFinalised(_auctionId, winner, paymentAmount);

    // NFT Transfer
     // If there's a winner, send them the NFT.
    if (winner != address(0)) {
        nftContract.safeTransferFrom(
            address(this),
            winner,
            auction.nftTokenId
        );

    // If no winner, return the NFT to the seller.
    } else {
        nftContract.safeTransferFrom(
            address(this),
            auction.seller,
            auction.nftTokenId
        );
    }
}

// Allows losing bidders to make their deposits claimable by
   transferring their deposits to withdrawable balance.
function claimLosingBid(
    uint256 _auctionId  // The auction they bid in.
) external {
    Auction storage auction = auctions[_auctionId];

    // Checks that the auction is over and the caller is not the
       winner.
    require(auction.isFinalised, "Auction not finalised");
    require(msg.sender != auction.highestBidder, "Winner cannot
       claim refund here");

    Bid storage bid = bids[_auctionId][msg.sender];
    uint256 amount = bid.deposit;

    require(amount > 0, "No deposit to claim");

    // Set deposit to 0 before adding to withdrawal list.
    // This prevents a re-entrancy attack where they could claim
       multiple times.
    bid.deposit = 0;

    // Credit the bidder's internal account.
    pendingWithdrawals[msg.sender] += amount;
}

// Allows any user to withdraw their balance from the contract.
// This is the "pull" part of the secure pull-over-push pattern.
function withdrawFunds() external{
```

```
        // Get the amount this user is owed.
        uint256 amount = pendingWithdrawals[msg.sender];

        require(amount > 0, "No funds to withdraw");

        // Set their internal balance to 0 before sending the Wei.
        // This is the most critical part of preventing re-entrancy
           attacks.
        pendingWithdrawals[msg.sender] = 0;

        // Send the Wei to the user.
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");

        emit FundsWithdrawn(msg.sender, amount);
    }
}
```